

Hochschule Darmstadt (h_da)

Fachbereich Informatik

Modul: Human-Computer Interaction, SoSe 2026

Betreuung: Prof. Dr. Kawa Nazemi

Meilenstein 3: Implementierung

GoTrip – KI-gestützte Gruppenreiseplanung

Gruppe 9

Mohamad Haj Ahmad · Wajdy Eleyan · Amal Najah · Eya Mathlouthi

Darmstadt, 20. Juli 2026

Inhaltsverzeichnis

1. Entwicklungs-Fundament	2
1.1 Haupt-Komponenten und Funktionen	2
1.2 Bibliotheken und Tools	2
1.3 Nutzungskontext und Ansichten	3
1.4 Datenquellen und API-Endpunkte	3
1.5 Code-Splitting und Ordnerstruktur	4
2. Implementierung	5
2.1 Systemübersicht	5
2.2 Code-Komentierung und Autorenschaft	5
2.3 README und Release-Notes	6

1. Entwicklungs-Fundament

1.1 Haupt-Komponenten und Funktionen

GoTrip ist eine mobile Web-App für die gemeinsame Gruppenreiseplanung. Die Anwendung besteht aus 16 Screens (Seiten), die jeweils einer eigenen Route entsprechen, sowie einer Reihe wiederverwendbarer Komponenten und Dienste. Die folgende Tabelle gibt einen Überblick über die wichtigsten Screens und deren Funktionen:

Tabelle 1: Haupt-Screens und ihre Funktionen

Screen	Funktion	Autorin / Autor
Login	Name eingeben, kein Account erforderlich	Eya Mathlouthi
Home	Alle eigenen Reisen auf einen Blick	Wajdy Eleyan
Reise erstellen	Name, Zeitraum (Wheel-Picker), Budget, Gruppe	Wajdy Eleyan
Freunde einladen	Einladungslink teilen via Web-Share-API	Wajdy Eleyan
Verfügbarkeit	Kalender mit 3 Zuständen (Verfügbar / Vielleicht / Nein)	Eya Mathlouthi
Präferenzen	Budget-Picker + 9 Interessen-Chips	Eya Mathlouthi
KI-Empfehlung	Reiseziele vorschlagen, analysieren, KI-Score abrufen	Mohamad Haj Ahmad
Übersicht (Management)	Gruppenübersicht, Mitgliederstatus, Schlussfolgerung	Wajdy Eleyan
Aktivitäten	Vorschläge erstellen und per Thumbs-up abstimmen	Amal Najah
Budget-Tracker	Ausgaben pro Mitglied erfassen und aufschlüsseln	Amal Najah
Reiseplan	Tagesitinerary mit Zeitslots	Amal Najah
Abstimmung	Reiseziele mit Sternebewertung bewerten	Amal Najah

1.2 Bibliotheken und Tools

Die folgende Tabelle listet alle verwendeten Bibliotheken und Tools auf. Neben React bilden React Router (Routing), die Context API (Zustandsverwaltung) und Tailwind CSS (Styling) das Kern-Framework der Anwendung:

Tabelle 2: Verwendete Bibliotheken und Tools

Bereich	Technologie / Bibliothek	Version	Zweck
UI-Framework	React + TypeScript	React 19	Komponentenbasiertes Frontend
Build-Tool	Vite	v8	Schneller Entwicklungsserver und Build
Styling	Tailwind CSS	v4	Utility-first CSS-Framework
Routing	React Router	v7	Client-seitiges Routing, 16 Routen
Zustandsverwaltung	React Context API	–	AuthContext, TripContext, LanguageContext, AccessibilityContext
Formulare	React Hook Form + Zod	–	Formularvalidierung mit Schema-Typisierung
Icons	lucide-react	–	Konsistentes Icon-Set
UI-Komponenten	shadcn/ui	–	Barrierefreie Basis-Komponenten
Backend	Express.js	–	REST-API, NASA-Proxy, DB-Synchronisation
Datenbank	PostgreSQL (Neon Cloud)	–	Persistenz, Echtzeit-Sync via HTTP-Polling
Deployment	Render.com	–	Hosting für Frontend und Backend
Internationalisierung	Eigene i18n-Lösung	–	Deutsch, Englisch, Spanisch

1.3 Nutzungskontext und Ansichten

GoTrip ist ausschließlich als **Mobile-Anwendung** konzipiert. Die App richtet sich an junge Erwachsene (Personas: Leon, 22 J.; Maya, 25 J.), die unterwegs in einer Gruppe eine Reise planen. Der Nutzungskontext ist mobil — auf dem Smartphone, häufig in der Gruppe oder in öffentlichen Verkehrsmitteln.

Die **Mobile-Ansicht hat uneingeschränkten Vorrang**. Das Layout verwendet eine feste maximale Breite von 430 px (entspricht einem Standard-Smartphone), eine fixierte Bottom-Navigation, native iOS-ähnliche Steuerelemente (Wheel-Date-Picker, Bottom-Sheets, Swipe-Gesten) und Touch-optimierte Schaltflächen. Eine Tablet- oder Desktop-Ansicht ist nicht vorgesehen; die App ist jedoch im Browser aufrufbar und skaliert auf größeren Bildschirmen innerhalb der mobilen Maximalbreite zentriert.

1.4 Datenquellen und API-Endpunkte

GoTrip nutzt mehrere externe Datenquellen für die KI-Score-Berechnung. Alle Anfragen an kostenpflichtige oder CORS-beschränkte APIs werden über den eigenen Express-Server

(`server/index.js`) als Proxy weitergeleitet:

Tabelle 3: Externe Datenquellen und API-Endpunkte

Datenquelle	API	Verwendungszweck	Kosten
Klimadaten (ERA5)	Open-Meteo Climate API	Temperatur, Niederschlag, Sonnenstunden	kostenlos
Küstenerkennung	Open-Meteo Marine API	Strandeignung: Meerestemperatur, Wellengang	kostenlos
Geocoding	Open-Meteo Geocoding API	Stadtname → Koordinaten	kostenlos
Biodiversität	GBIF Occurrence API	Artenvielfalt am Reiseziel	kostenlos
Wetterdaten (NASA)	NASA POWER API	Historische Sonneneinstrahlung	kostenlos
Satellitendaten	NASA Earthdata MODIS	Landnutzung, Vegetationsindex	Token erforderlich

Die Score-Berechnung kombiniert Klimakomfort (40 %) und Interessenerfüllung (60 %):

$\text{Gesamtscore} = \text{Klimakomfort} \times 0,4 + \text{Interessenerfüllung} \times 0,6$

Klimakomfort (ERA5):

Temperatur 20-26 °C → +25 Punkte

Niederschlag < 20 mm → +12 Punkte

Sonne ≥ 9 h/Tag → +8 Punkte

Interessenerfüllung (je nach gewähltem Interesse):

Strand (beach) → küstennahe Stadt? Ja → 85-95 Pkt., Nein → 8 Pkt.

Stadt (city) → Einwohner ≥ 500.000? Ja → 92 Pkt., Nein → 18 Pkt.

Natur (nature) → GBIF-Arten > 1.000.000? Ja → 85 Pkt., Nein → 55 Pkt.

Hinweis: Der NASA-Earthdata-Token wird in `server/.env` als Umgebungsvariable (`EARTHDATA_TOKEN`) gespeichert und ist in `.gitignore` aufgeführt — er wird nie in das Repository committed.

1.5 Code-Splitting und Ordnerstruktur

Die Anwendung ist in drei Hauptbereiche aufgeteilt: Frontend (`src/`), Backend (`server/`) und Dokumentation (`docs/`). Jede der 16 Seiten liegt in einer eigenen Datei unter `src/pages/`, was eine klare Trennung nach Screens ermöglicht (dateibasiertes Code-Splitting). Gemeinsam genutzte Komponenten sind nach funktionalem Bereich gruppiert:

```

gotrip/
├── src/
│   ├── App.tsx           Routing-Konfiguration (16 Routen)
│   ├── main.tsx          Einstiegspunkt
│   ├── components/
│   │   ├── activities/   ActivityCard
│   │   ├── budget/       AddExpenseForm, BalanceSummary, ExpenseCard
│   │   ├── calendar/     AvailabilityCalendar, GroupHeatmap
│   │   ├── members/      InviteLink, MemberCard, MemberList
│   │   ├── preferences/  BudgetSlider, InterestChips, PreferencesForm
│   │   ├── recommendations/ DestinationCard, ScoreRing, DataSourceBadges
│   │   ├── shared/       BottomNav, BottomSheet, PageHeader, Toast
│   │   ├── sheets/       10 Bottom-Sheet-Dialoge
│   │   ├── trips/        TripCard, TripForm
│   │   ├── ui/           Button, Card, Input, WheelDatePicker
│   │   └── voting/       StarRating, VotingCard
│   ├── context/
│   │   ├── AuthContext.tsx  Nutzersession (Name, Trip-ID)
│   │   ├── TripContext.tsx  Reise-Zustand (Mitglieder, Ziele, Budget)
│   │   ├── LanguageContext.tsx Sprachwahl (DE / EN / ES)
│   │   └── AccessibilityContext.tsx Barrierefreiheits-Einstellungen
│   ├── i18n/
│   │   └── translations.ts  Übersetzungen für DE, EN, ES
│   ├── lib/
│   │   └── utils.ts         Hilfsfunktionen (cn, Datumsformat)
│   ├── pages/              16 Screen-Dateien (je eine Route)
│   └── services/
│       ├── llmService.ts    Koordiniert alle API-Aufrufe
│       ├── realData.ts      Anbindung: Open-Meteo, GBIF, NASA POWER
│       ├── scoreEngine.ts   Regelbasierte Score-Berechnung
│       ├── tripSync.ts      HTTP-Polling alle 6 s (Echtzeit-Sync)
│       ├── activitiesService.ts Aktivitäten-CRUD
│       ├── account.ts       Nutzer-Verwaltung
│       └── joinTrip.ts      Beitritt per Einladungslink
└── server/
    ├── index.js            Express: DB-Endpunkte + NASA-Proxy
    └── .env                DATABASE_URL, EARTHDATA_TOKEN (nicht committet)

```

2. Implementierung

2.1 Systemübersicht

GoTrip wurde als Progressive Web App (PWA) mit React 19 und TypeScript implementiert. Das Frontend kommuniziert mit dem Express-Backend über REST-Endpunkte. Für die Echtzeit-Synchronisation zwischen Gruppenmitgliedern wird HTTP-Polling im 6-Sekunden-Intervall

eingesetzt (src/services/tripSync.ts). Die Datenbank liegt in der Cloud (Neon PostgreSQL); das Backend läuft auf Render.com.

Tabelle 4: Systemarchitektur im Überblick

Schicht	Technologie	Beschreibung
Frontend	React 19 + TypeScript + Vite 8	SPA, 16 Routen, Mobile-first
Styling	Tailwind CSS v4	Utility-first, max-width 430 px
State Management	React Context API	4 Contexts (Auth, Trip, Language, Ally)
Backend	Express.js (Node.js)	REST-API, NASA-Proxy, DB-Sync
Datenbank	PostgreSQL – Neon Cloud	Trips, Members, Votes, Expenses
Echtzeit	HTTP-Polling (6 s)	Gruppenänderungen in Echtzeit sichtbar
Deployment	Render.com	Frontend + Backend als ein Service
Live-URL	gotrip-9id7.onrender.com	

2.2 Code-Kommentierung und Autorenschaft

Jede Quelldatei enthält im Dateikopf einen Autor-Kommentar sowie inline erläuternde Kommentare auf Deutsch. Das folgende Beispiel zeigt das verwendete Kommentarschema:

```
// Autor: Mohamad Haj Ahmad
// src/services/scoreEngine.ts
// Regelbasierte Berechnung des KI-Scores für ein Reiseziel.
// Score = Klimakomfort (40 %) + Interessenerfüllung (60 %)

export function calculateScore(climate: ClimateData, interests: string[]): number {
  // Klimakomfort: Temperatur, Niederschlag, Sonnenstunden auswerten
  const climateScore = evaluateClimate(climate);

  // Interessenerfüllung: je Interesse einen Teilscore berechnen
  const interestScore = evaluateInterests(interests, climate);

  // Gewichtete Summe zurückgeben
  return climateScore * 0.4 + interestScore * 0.6;
}
```

Die folgende Tabelle zeigt die Verantwortlichkeiten auf Dateiebene:

Tabelle 5: Autorenschaft nach Bereich

Bereich / Dateien	Autorin / Autor
Backend, Datenbank, API-Integration, Score-Engine server/index.js · services/realData.ts · services/scoreEngine.ts · services/IlmService.ts	Mohamad Haj Ahmad
Routing, TripDashboard, Management, Sync-System, Reise erstellen App.tsx · pages/Home · pages/Management · services/tripSync.ts · pages/CreateTrip	Wajdy Eleyan
UI-Komponenten, DestinationCard, Aktivitäten, Budget-Tracker, Reiseplan components/recommendations/ · pages/ActivitiesVoting · pages/BudgetTracker · pages/FinalOverview	Amal Najah
StarRating, Präferenzen, Verfügbarkeit, Login, Voting components/voting/ · pages/Preferences · pages/Availability · pages/Login · pages/VoteDecide	Eya Mathlouthi

2.3 README und Release-Notes

Die README-Datei (README.md im Projektstamm) enthält alle geforderten Abschnitte: Projektbeschreibung, Tech-Stack, Funktionsübersicht, vollständige Installationsanleitung sowie einen Verweis auf die Release-Notes. Die Release-Notes dokumentieren alle Versionen mit Datum und Verantwortlichkeiten:

Tabelle 6: Release-Notes – Übersicht

Version	Datum	Änderungen	Verantwortlich
v1.0.0	28.06.2026	Initiales Release: Login, Home, Reise erstellen, Einladungslink, Verfügbarkeitskalender, Präferenzen, PostgreSQL-Anbindung	Alle
v1.1.0	05.07.2026	KI-Empfehlung: Open-Meteo, GBIF, NASA POWER integriert; Score-Engine; DestinationCard mit ScoreRing; DataSourceBadges	Mohamad, Amal
v1.2.0	12.07.2026	Aktivitäten-Voting, Budget-Tracker, Reiseplan, Mehrsprachigkeit (DE/EN/ES), HTTP-Polling für Echtzeit-Sync	Amal, Wajdy
v1.3.0	20.07.2026	WheelDatePicker (iOS-Stil), StarRating mit eindeutigen clipPath-IDs, KI-Score dynamisch aus Präferenzen, Bugfixes Navigation	Eya, Wajdy

Installationsanleitung (Kurzfassung aus README.md):

```
# 1. Repository klonen
git clone https://github.com/mo518525/Trip-Planer.git
cd Trip-Planer

# 2. Frontend starten
npm install
npm run dev          # → http://localhost:5173

# 3. Backend starten
cd server
npm install
node index.js        # → http://localhost:3001

# 4. Umgebungsvariablen (server/.env)
DATABASE_URL=postgresql://...      # Neon PostgreSQL Connection String
EARTHDATA_TOKEN=...                # NASA Earthdata Bearer Token (optional)
```

Abgabehinweis: Der Ordner `node_modules/` ist in `.gitignore` aufgeführt und wird nicht abgegeben. Große Datensätze (> 200 MB) werden nicht verwendet — alle externen Daten werden zur Laufzeit über die oben aufgeführten APIs abgerufen.